Enhancing Pre-trained Diffusion Models with Reinforcement Learning and Adversarial Reward Functions

Qicheng Xu
IIIS, Tsinghua University
xuqc24@mails.tsinghua.edu.cn

Zehua Wang
IIIS, Tsinghua University
wangzehu24@mails.tsinghua.edu.cn

Abstract

Diffusion models have emerged as powerful generative tools but remain limited by their fixed training objectives that focus on noise prediction. In this project, we introduce a novel approach that enhances pre-trained diffusion models through reinforcement learning (RL) with adversarial reward functions. By formulating the reverse diffusion process as a Markov Decision Process and leveraging GAN-inspired discriminators to provide per-timestep rewards, our method optimizes diffusion models for generation quality without altering their architecture. Experiments on image generation demonstrate that our approach improves quality (FID scores) and even sample efficiency, compared to the base model, with the EMA variant showing consistent stability and performance improvements. Our framework provides a plug-and-play enhancement for existing diffusion models that can be directly applied to downstream tasks without requiring additional inference-time computation. Our code is available at https://github.com/HACLINE/DiffPPOGAN.

1. Introduction

Diffusion models have achieved remarkable success in various generative tasks, including image synthesis, text-to-image generation, and video generation. However, the convolutional structure and fixed mathematical formulation of diffusion models limit their abilities in pure image generation [11]. Previous works have attempted to enhance the performance of diffusion models by distillation [8, 15, 23] or internal supervised loss [22].

In this work we consider enhancing the performance of diffusion models, without changing the process, by introducing reinforcement learning (RL) and adversarial reward functions. By viewing the diffusion process as a Markov Decision Process (MDP), we can apply RL techniques to optimize the diffusion model. Since the overall diffusion process remains unchanged, models from our approach can be directly applied to downstream tasks, which is an enhancement at no cost. Furthermore, our approach provides a more direct optimization towards overall perceptual quality rather than solely minimizing pixel-wise mean squared error between noise predictions.

The goal of this work is to prove the effectiveness of RL in training diffusion models and the idea of designing reward functions with adversarial discriminators, and to propose a new free lunch for diffusion models and their downstream tasks or pipelines (*e.g.*, distillation).

2. Related Work

Diffusion Models. Diffusion models have emerged as a powerful class of generative models. Denoising Diffusion Probabilistic Models (DDPM) [7] established the standard training framework for diffusion models by gradually denoising Gaussian noise through a learned reverse process. The Denoising Diffusion Implicit Models (DDIM) [19] formulated the diffusion process with more flexible training and sampling schedules, and it is well known for its deterministic sampling process.

Reinforcement Learning for Generation. Reinforcement learning, originally developed for game play or robotic decision-making, has shown remarkable effectiveness in various generative domains. In language modeling, autoregressive models are trained using human preference data [14]. Recently, post-training alignment through RL has emerged as a promising paradigm, enabling LLMs to have better reasoning capabilities [18]. Within the visual domain, RL approaches have enabled stroke-based image synthesis [4] and refinement of outputs from pretrained generative models [1]. More recently, Black *et al*. [2] demonstrated that RL can be used to train diffusion models directly, though they primarily focused on text-to-image alignment rather than improving the general quality of the diffusion process.

GAN and Hybrid Approaches. Generative Adversarial Networks (GANs) [5] introduced the adversarial training paradigm where a generator and discriminator compete in

a minimax game. Although GANs can produce sharp images, they often suffer from training instability and mode collapse. Several attempts have been made in literature to combine GANs with diffusion models, such as Diffusion-GAN [21], which incorporates the forward diffusion process with adversarial training.

3. Methods

We propose a reinforcement learning framework for fine-tuning diffusion models, combining the strengths of both paradigms to enhance the quality of image generation. Our approach addresses the limitations identified in the introduction 1 by providing a direct optimization path toward perceptual quality rather than just noise prediction accuracy. An overview of our framework is shown in Figure 1.

3.1. MDP Formulation

Reverse diffusion process shows a denoising trajectory $x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$. We retain the first $T - T'$ steps to generate the noisy image $x_{T'}$ following the original distribution, and formalize the rest process $x_{T'} \rightarrow x_{T'-1} \rightarrow \cdots \rightarrow x_0$ as a fixed-horizon Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, r, \mathcal{T})$ where:

- **States** s_t correspond to noisy images $(\mathbf{x}_t, t)$ at diffusion timestep t for $t = 0, 1, \ldots, T'$.
- **Actions** a_t parameterize the predicted noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ removed at each step.
- **Transition function** $\mathcal{T}(s_{t-1}|s_t, a_t)$ follows the diffusion sampling process, which is DDIM, a deterministic process in our case.
- **Rewards** Inspired by Wang *et al*. [21], the dense reward $r(s_t, a_t)$ depends on a learned discriminator $D(\mathbf{x}_{t-1}, t-1)$ and intermediate density estimates.

By retaining first $T-T'$ steps to generate the noisy image $x_{T'}$, the model effectively prevents collapse into a single optimal trajectory, thereby preserving the diversity of generated images. This MDP formulation allows us to leverage the rich literature of reinforcement learning optimization while keeping the diffusion sampling structure intact.

3.2. Reward Function Design and Alternatives

As opposed to directly backpropagating through the discriminator loss and providing gradient information [21], we use the discriminator to provide a reward signal for the RL agent. This design allows us to separate the gradients of training the discriminator and updating the diffusion model, potentially alleviating the instability issues associated with the manifold hypothesis.

The reward function is critical to our approach and consists of two key components:

- **Discriminator-based rewards**: A trained discriminator $D(\mathbf{x}_t, t)$ evaluates the realism of intermediate states, providing per-timestep evaluation.
- **Time-weighted importance**: We apply a reward scheduler such that the more noisy image has a smaller reward guidance: $r_t = w_t^r \cdot (2D(\mathbf{x}_t, t) - 1) \in (-w_t^r, w_t^r)$.

This design reflects two important insights: (1) timesteps with less noise produce more visually meaningful images that better indicate final quality, and (2) the discriminator provides more reliable signals on less noisy images, reducing variance in the reward signal.

We also explored alternative reward formulations. In particular, we experimented with unbounded rewards in $\mathbb{R}$. However, this led to rapid scaling up, destabilizing training, and resulting in worse performance. The bounded reward design we ultimately adopted provides stability while maintaining the informative signal from the discriminator.

3.3. Discriminator Optimization

We use a weighted version of the original discriminator loss in GAN [5]:

$$\begin{aligned}\mathcal{L}_D(\phi) =& \mathbb{E}_t \left[w_t^d \cdot \mathbb{E}_{\substack{x_{T'} \sim p_{T'} \\ x_t \sim \text{Denoise}(x_{T'}, \epsilon_\theta, t)}} \left[\log D_\phi(x_t, t) \right] \right] \\ &+ \mathbb{E}_t \left[w_t^d \cdot \mathbb{E}_{\substack{x_0 \sim \mathcal{D}_{\text{real}} \\ x_t \sim \text{AddNoise}(x_0, t)}} \left[\log \left(1 - D_\phi(x_t, t) \right) \right] \right].\end{aligned} \tag{1}$$

Here, $p_{T'}$ is the probability distribution of $x_{T'}$ generated by the base diffusion model. w_t^d is a weight that depends only on timesteps, with smaller values for larger timesteps. This design follows the insight that being strict about distinguishing noisy images is fruitless and inaccurate.

Drawing from recent advances in GAN stability studied in the course, we leverage gradient penalty regularization techniques from R3GAN [9]. While the RpGAN component [10] of R3GAN did not prove useful in our setting, the R_1 and R_2 regularizations significantly improved training stability, which was crucial given the complex interplay between reinforcement learning and adversarial training.

Specifically speaking, we implement the gradient penalty regularization techniques proposed by Mescheder *et al*. [13]. We apply R_1 and R_2 regularization terms in diffusion case:

$$R_1(\phi) = \frac{\gamma}{2} \mathbb{E}_t \left[w_t^d \mathbb{E}_{\substack{x_0 \sim \mathcal{D}_{\text{real}} \\ x_t \sim \text{AddNoise}(x_0, t)}} \left[\|\nabla_x D_\phi(x_t, t)\|^2 \right] \right] \tag{2}$$

$$R_2(\phi) = \frac{\gamma}{2} \mathbb{E}_t \left[w_t^d \mathbb{E}_{\substack{x_{T'} \sim p_{T'} \\ x_t \sim \text{Denoise}(x_{T'}, \epsilon_\theta, t)}} \left[\|\nabla_x D_\phi(x_t, t)\|^2 \right] \right] \tag{3}$$

where γ is a hyperparameter controlling the strength of the regularization. The R_1 penalty is applied to real data samples, while R_2 is applied to generated samples. These gradient penalties help stabilize training by enforcing Lipschitz constraints on the discriminator, preventing aggressive updates that can destabilize the GAN training dynamics. In

Figure 1. Overview of our framework. We formulate the reverse diffusion process as a Markov Decision Process (MDP) and use an adversarial discriminator to provide per-timestep rewards. The diffusion model is trained using Proximal Policy Optimization (PPO . The discriminator is trained using a weighted version of the original GAN loss, with gradient penalties to stabilize training.

contrast, other types of gradient penalties, such as the one-centered gradient penalty (1-GP) in WGAN-GP [6], do not offer convergence guarantees. [9]

3.4. RL Optimization

For policy optimization, we employ Proximal Policy Optimization (PPO) [17] due to:

- Its ability to handle continuous action spaces (noise predictions in our case)
- Sample efficiency compared to other policy gradient methods
- Stability through trust region constraints that prevent destructive policy updates

Here are several keypoints in our implementation, either to stabilize training or to clarify details not mentioned before:

- The key difference between DDPM and DDIM inference in this problem is the determinacy of the environment transition. Here we choose DDIM, corresponding to a fully deterministic environment, both for stabilizing training and for fine-tuning does not need much exploration but exploitation after some exploration.
- Since PPO needs a nondeterministic action distribution, we perturb the predicted noise (action) at a small fixed scale σ while preserving the norm of predicted noise:

$$\epsilon_{\text{perturb}} = \sqrt{1-\sigma^2}\epsilon + \sigma\epsilon_{\text{noise}}, \epsilon_{\text{noise}} \sim \mathcal{N}(0, I). \quad (4)$$

- For stable training, we apply a normalization across the batch of advantages $\{A_t\}_{\mathcal{B}}$ to shift the mean to 0 and scale the variance to 1. This normalization is crucial for PPO, as it helps avoid scale issues that can cause instability during training.
- For stable training, as applied by Tang *et al*. [20], we track the Kullback-Leibler Divergence (KL divergence) [12] of the current model from the one when trajectories are sampled. Once an upper bound has been exceeded, the model will not be updated until new trajectories are sampled.

3.5. Freezing Discriminator

During adversarial training, the discriminator is usually more powerful than the generator, leading to a situation where the generator fails to learn effectively and is stuck in local minima. To address this issue, we implement a freezing mechanism for the discriminator. This involves freezing the discriminator's parameters when the gap between the real images' validity and the generated images' validity is larger than a certain threshold (*e.g*., 0.3). This allows the generator to catch up and learn from the discriminator's feedback without being overwhelmed by its strength. When the validity of the generated images is higher than that of real images, the discriminator is unfrozen and allowed to continue training. This mechanism helps to dynamically balance the training of the generator and the discriminator, preventing the generator from being stuck in local minima and ensuring that both components of the GAN are able to learn effectively.

3.6. Reset Technique

During our experiments, we observed that the FID score could rapidly deteriorate. To mitigate this issue, we implemented a reset mechanism that monitors the FID-5k scores every few epochs and resets the model to the best checkpoint if the current score exceeds the best score by a predetermined threshold (*e.g.*, 1.0). This technique is proved essential for maintaining consistent improvements throughout training, while still allowing for the exploration of new trajectories. Ablation studies in Section 4.5.1 demonstrate the effectiveness of the reset technique.

3.7. Exponential Moving Average (EMA) Model

We employ an Exponential Moving Average (EMA) model to stabilize the training process. The EMA model is updated using the following equation:

$$\theta' \leftarrow \beta\theta' + (1-\beta)\theta \tag{5}$$

where β is a hyperparameter controlling the decay rate of the EMA. The EMA model is used for sampling during evaluation, as it tends to produce more stable and high-quality samples compared to the current model. This is particularly important in the context of diffusion models, where the generated samples can be sensitive to small changes in the model parameters.

An overview of the training process is shown in Algorithm 1 below. Our method synthesizes concepts from both reinforcement learning and adversarial training courses, applying them to the unique challenges of diffusion model optimization. The combination of PPO's stable policy updates, discriminator-based rewards, and regularization techniques from modern GAN literature creates a robust framework for enhancing pretrained diffusion models without modifying their architecture or inference process.

Algorithm 1 Training

Input: Pretrained diffusion model ϵ_θ, real data distribution $\mathcal{D}_{\text{real}}$
Initialize: Discriminator D_ϕ, value function V_ψ, EMA model $\epsilon_{\theta'}$
for iteration $i = 1, 2, ...$ **do**
  Sample batch of real images $x_0 \sim \mathcal{D}_{\text{real}}$
  Sample $x_{T'} \sim p_{T'}$ using the base diffusion model
  Generate trajectories by denoising $x_{T'}$ using ϵ_θ
  Compute rewards $r_t = w_t^r \cdot (2D_\phi(x_t, t) - 1)$
  Update discriminator D_ϕ using Equation 1 adding R_1(Equation 2) and R_2(Equation 3) regularization
  Update policy ϵ_θ using PPO with KL constraint
  Update EMA model $\epsilon_{\theta'}$
  if FID score increases by more than threshold **then**
    Reset $\epsilon_\theta, V_\psi, D_\phi$ to best checkpoint
  end if
end for

4. Experiments

Our implementation uses PyTorch, and is based on the open-source code of Generative Zoo [3]. The code of our project is available at https://github.com/HACLINE/DiffPPOGAN.

4.1. Model Architectures

We implement three key architectural components as follows:

- **Diffusion Model**: U-Net architecture [16] with skip connections to capture multiscale features, following standard practices in diffusion modeling.
- **Discriminator**: Adapted from the timestep-embedded encoder of the U-Net with an additional classification head to predict image realism conditioned on timestep.
- **Critic Network**: Similar to the discriminator but modified to produce real-valued value estimates rather than binary classifications, facilitating the learning of value functions.

4.2. Evaluation

We mainly evaluate our approach using the FID score, which measures the distance between the feature distributions of the generated and real images. Lower FID scores indicate higher quality and diversity.

4.3. Experiment Details

Our base diffusion model is a small one that was trained strictly following DDPM with the number of total timesteps to $T = 1000$. We don't use open-source pretrained model because we failed to find pure DDPM diffusion model of appropriate size suitable for experiments. In practice, to facilitate iterative testing, we use the model as $T = 100$.

The hyperparameters are set as follows. We set the number of training timesteps to $T' = 400$ (see Section 3.1 for definitions) and $w_t^d = w_t^r = \bar{\alpha}_t$, where $\bar{\alpha}_t$ follows the definition of the original DDPM paper [7], $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t}(1-\beta_s)$, and β_s denotes the variance of the forward diffusion process. In particular, we do fast sampling for the first $T - T'$ steps, the timesteps without RL training, which is a standard practice in diffusion models to trade off sample quality for speed with base model.

For our experiments, we only use the CIFAR-10 dataset, as the training process and iterative experimentation are both time and resource-intensive. We adopt the standard train/test split provided with CIFAR-10, and compute the

FID score over the entire set of images.

More details of the experiment can be found in the Appendix B and on the project page.

4.4. Experiment Results

Our base model reaches an FID score of 18.26 for $T = 100$ and 15.75 for $T = 1000$. Our approach achieves an FID score of **13.11** for $T = 100$ and 16.81 for $T = 1000$. Notably, our approach with $T = 100$ has a lower FID score than the base model with $T = 1000$, indicating that our approach is able to improve both sample efficiency and sample quality.

We also observe that the FID score of our approach with $T = 1000$ is higher than that with $T = 100$, which is likely due to the fact that the first $T - T'$ steps are faster and less accurately sampled with the base diffusion model.

We show some generated samples in Figure 2. The images on the left are generated using the base diffusion model, while those on the right are generated using our approach. Since the first $T - T'$ steps are the same, what our approach actually does is to improve the shadow and smoothness of the generated images, instead of changing the main content of the images.

4.5. Ablation Studies

We conduct ablation studies to evaluate the impact of different components in our framework.

4.5.1. Reset Technique

In Figure 3, we observe that the reset technique significantly improves the stability and performance of our approach. Without the reset technique, the FID-5k scores increase rapidly and the model collapses, generating images that prefer to be simple color blocks for background.

4.5.2. R_1 and R_2 Regularization

In Figure 5, by observing the EMA FID-5k score, we find that R_1 and R_2 regularization improves the performance of our approach. Although R1 and R2 regularization actually influences the stability of the training process, we find that together with the reset technique and the EMA model, the training process can be stabilized and the performance can be improved.

Figure 5a also shows the ability of EMA model in stabilizing the training process. We observe that the FID-5k scores with the EMA model have smaller variance and are lower than those without the EMA model. This indicates that the EMA model can effectively stabilize the training process and improve the performance of our approach.

4.5.3. Trained Timesteps

Although models are used to sample images with total timesteps $T = 100$, we still train the model as $T' = 400\,(T = 1000)$ instead of $T' = 40\,(T = 100)$. In Figure 6, we find that the model trained with $T = 1000$ has lower FID-5k scores than the one trained with $T = 100$. This can be explained by higher sample efficiency and the generalizability of the model across different timesteps.

5. Conclusion

In this paper, we introduced a novel framework that enhances pretrained diffusion models using reinforcement learning with adversarial reward functions. By formulating the diffusion process as an MDP and applying PPO with a discriminator-based reward function, we demonstrated improvements in FID scores without changing the model architecture or inference process. Because our approach does not require additional inference-time computation and does not change the diffusion process, it can be easily applied to existing distillation pipelines and downstream tasks as a free plug-and-play enhancement.

Our work highlights several key insights:

- Reinforcement learning provides an effective mechanism for optimizing diffusion models toward perceptual quality metrics rather than just noise prediction.
- Time-weighted discriminator rewards offer a valuable training signal that adapts to different levels of image fidelity during the denoising process.
- The combination of EMA, gradient penalties, and KL-constrained updates significantly stabilizes the notoriously difficult joint training of diffusion models, RL algorithms, and adversarial networks.

Future work could explore several promising directions:

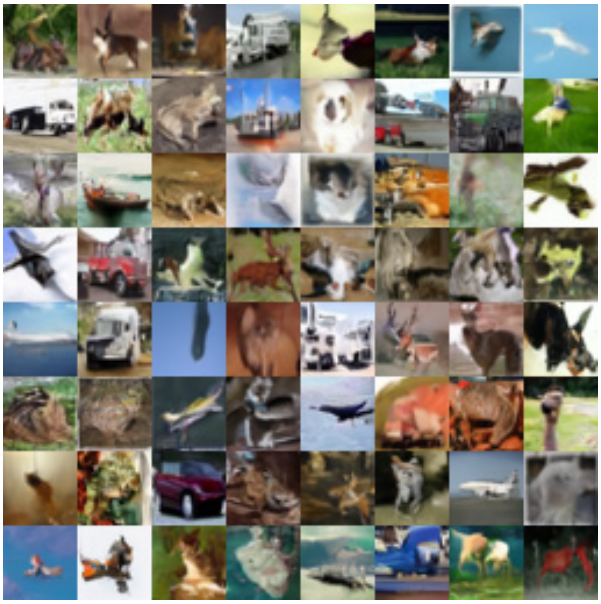
- Extending our approach to conditional generation tasks such as text-to-image or image-to-image translation.
- Investigating more sophisticated reward functions that incorporate multiple perceptual metrics beyond the GAN discriminator.
- Exploring how our reinforcement learning framework could enable diffusion models to better maintain global coherence and semantic consistency in complex generation tasks.
- Guiding RL exploration with real trajectories generated from real data, potentially improving sample efficiency and quality.

References

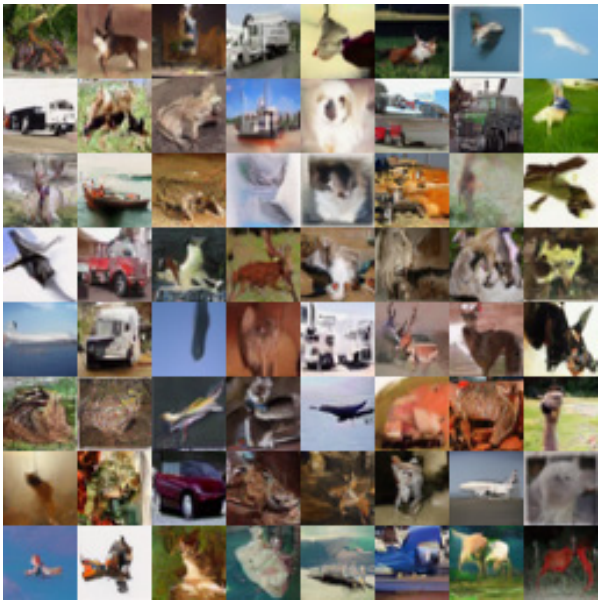
[1] Lars Ankile, Anthony Simeonov, Idan Shenfeld, Marcel Torne, and Pulkit Agrawal. From imitation to refinement – residual rl for precise assembly, 2024. 1

[2] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning, 2024. 1

[3] Francisco Caetano. A collection of generative algorithms and techniques implemented in python. https://github.com/caetas/GenerativeZoo, 2024. 4



(a) Samples with the base model.



(b) Samples from the same noises with our approach.

Figure 2. Sample images generated by the base model (left) and our approach (right). The images on the left are generated using the base diffusion model, while those on the right are generated using our approach. Both images are sampled from the same noise.

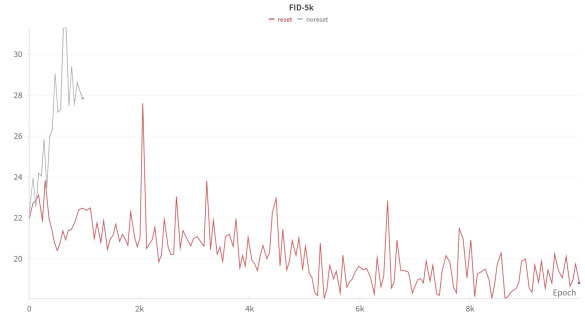


Figure 3. FID-5k scores with (Red) and without (Gray) the reset technique. The reset technique significantly improves the stability and performance of our approach.

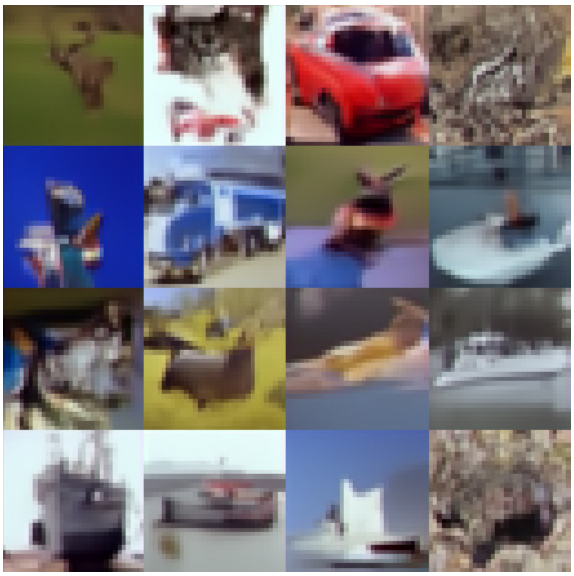


Figure 4. Sample images generated without the reset technique. We observe that, without the reset technique, the model collapses and generates images that are simple color blocks.

[4] Yaroslav Ganin, Tejas Kulkarni, Igor Babuschkin, S. M. Ali Eslami, and Oriol Vinyals. Synthesizing programs for images using reinforced adversarial learning, 2018. 1

[5] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014. 1, 2

[6] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans, 2017. 3

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. 1, 4

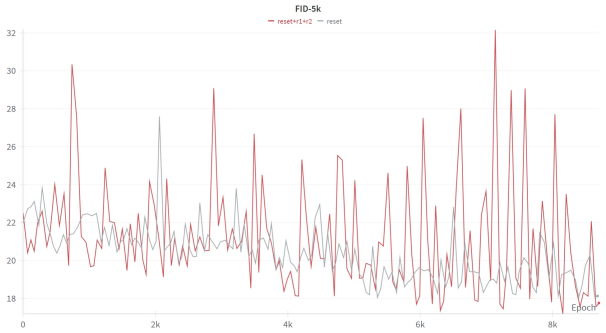
[8] Yi-Ting Hsiao, Siavash Khodadadeh, Kevin Duarte, Wei-An Lin, Hui Qu, Mingi Kwon, and Ratheesh Kalarot. Plug-and-play diffusion distillation, 2024. 1

[9] Yiwen Huang, Aaron Gokaslan, Volodymyr Kuleshov, and James Tompkin. The gan is dead; long live the gan! a modern gan baseline, 2025. 2, 3

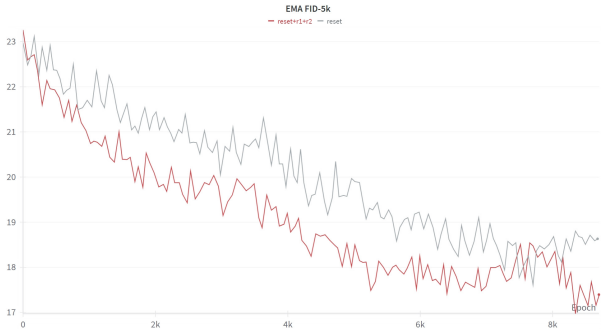
[10] Alexia Jolicoeur-Martineau. The relativistic discriminator: a key element missing from standard gan, 2018. 2

[11] Mason Kamb and Surya Ganguli. An analytic theory of creativity in convolutional diffusion models, 2024. 1

[12] Solomon Kullback and Richard A Leibler. On information



(a) FID-5k scores with (Red) and without (Gray) the R1 and R2 regularization, both with reset technique.



(b) EMA FID-5k scores with (Red) and without (Gray) the R1 and R2 regularization, both with reset technique.

Figure 5. FID-5k scores with (Red) and without (Gray) the R1 and R2 regularization, both with reset technique. The left figure shows the FID-5k scores, while the right figure shows the EMA FID-5k scores. Both figures are based on the same set of experiments.

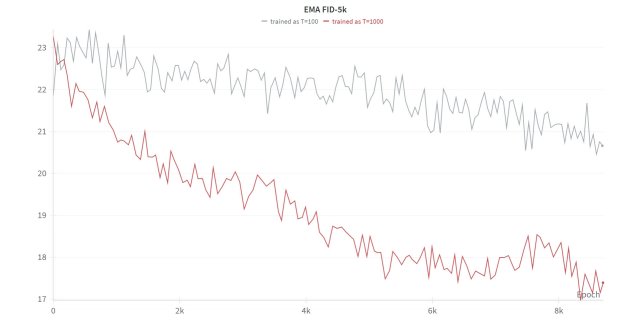


Figure 6. EMA FID-5k scores of models trained with $T = 1000$ (Red) and $T = 100$ (Gray). The model trained with $T = 1000$ has lower FID-5k scores than the one trained with $T = 100$.

and sufficiency. *The annals of mathematical statistics*, 22(1): 79–86, 1951. 3

[13] Lars Mescheder, Andreas Geiger, and Sebastian Nowozin. Which training methods for gans do actually converge?, 2018. 2

[14] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. 1

[15] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation, 2024. 1

[16] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. 4

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. 3

[18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. 1

[19] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022. 1

[20] Bingjie Tang, Iretiayo Akinola, Jie Xu, Bowen Wen, Ankur Handa, Karl Van Wyk, Dieter Fox, Gaurav S. Sukhatme, Fabio Ramos, and Yashraj Narang. Automate: Specialist and generalist assembly policies over diverse geometries, 2024. 3

[21] Zhendong Wang, Huangjie Zheng, Pengcheng He, Weizhu Chen, and Mingyuan Zhou. Diffusion-gan: Training gans with diffusion. *arXiv preprint arXiv:2206.02262*, 2022. 2

[22] Zhisheng Xiao, Karsten Kreis, and Arash Vahdat. Tackling the generative learning trilemma with denoising diffusion gans, 2022. 1

[23] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation, 2024. 1

A. Motivation

In diffusion models, if nerual networks have infinite capacity, the model should generate images exactly in training set. But this isn't true due to spatial properties and lipschitz limitation of convolutional neural networks. Also no one can guarantee that the model can generate meaningful images.

Later distillation on diffusion models use human-designed workflows to guide the model in various ways. Still the model is not guaranteed to generate meaningful images and should generate images in training set if having infinite capacity. They achieve better results than diffusion models because lipschitz limitation are relaxed.

Then why don't we use some self-supervised learning. We don't force the model to learn some human-designed goals, but let the model explore and exploit using RL. The model should find a way that it fits to generate images. After emergence of this idea, using an adversarial reward function is a natural choice, which is a good way to tell whether an image is visually meaningful or not.

B. More Experiment Details

You may find the implementation details of the experiments in this section. If you fail to find the details you are looking for, please look at the code repository https://github.com/HACLINE/DiffPPOGAN. We provide the configuration of the training process and models in Table 1 and Table 2, respectively.

Table 1. Configuration of training process.

	Parameter	**Value**	**Remark**
Training	Model learning rate	2e-4	
	Critic learning rate	5e-4	
	Discriminator learning rate	5e-4	
	Weight decay	1e-7	
	Batch size	128	
	Sample batch size	32	Number of trajectories for each rollout
	EMA rate	0.999	
	Number of epochs	10000	
Reset	Reset FID gap	1.0	Reset model when FID has risen by this value
	Cool down epochs	20	Number of epochs train only critic after reset
	Reset discriminator	true	Reset discriminator if model is reset
Diffusion	σ	2e-2	Noise of action(predicted noise)
	DDPM	0.0	Interpolation between DDPM and DDIM, 0.0 means DDIM
	$\beta_{start}, \beta_{end}$	1e-4 0.02	Linear schedule of noise
	T	1000	Number of total diffusion steps
	T'	400	Number of fine-tuning diffusion steps
	$T - T'$ sample timesteps	100	Number of total steps for first $T - T'$ steps sampling
	Sample timesteps	100	Number of total steps for sampling
PPO	Value loss clip	0.1	Clip bound of value loss
	Value loss coefficient	1	
	Clip value loss	true	
	Number of epochs train only value	20	
	KL loss Coefficient	0	
	Target KL	0.1	Target KL divergence in one epoch
	Log probability clip	0.2	Clip bound of log probability
	γ	0.997	Discount factor
	λ	0.95	Advantage smooth factor
Discriminator	Real fake validity gap	0.3	Freeze discriminator when validity gap is larger than this value
	γ	0.02	Coefficient for r1 r2 regularization

Table 2. Configuration of models.

	Parameter	**Value**
Diffusion Model	image_size	32
	in_channels	3
	model_channels	64
	out_channels	3
	num_res_blocks	2
	attention_resolutions	[4]
	dropout	0.0
	channel_mult	[1, 2, 2]
	conv_resample	true
	dims	2
	num_classes	null
	use_checkpoint	false
	use_fp16	false
	num_heads	4
	num_head_channels	32
	num_heads_upsample	-1
	use_scale_shift_norm	false
	resblock_updown	false
	use_new_attention_order	false
Critic Model	in_channels	3
	model_channels	64
	num_res_blocks	2
	attention_resolutions	[4]
	image_size	32
	dropout	0.0
	channel_mult	[1, 2, 2]
	conv_resample	true
	dims	2
	use_checkpoint	false
	num_heads	4
	num_head_channels	32
	use_scale_shift_norm	false
Discriminator Model	in_channels	3
	model_channels	64
	num_res_blocks	2
	attention_resolutions	[4]
	image_size	32
	dropout	0.0
	channel_mult	[1, 2, 2]
	conv_resample	true
	dims	2
	use_checkpoint	false
	num_heads	4
	num_head_channels	32
	use_scale_shift_norm	false